INGENIERÍA DE SOFTWARE II · PRIMER CORTE · 2026.2

Banco de Preguntas Saber Pro

Aplicación de escritorio en Java para crear, validar y revisar preguntas de selección múltiple.

Johan Santiago Lopez Ortega **Sebastian Tintinago Pantoja** **Eddy Emmanuel Sanchez Obando**

Universidad del Cauca · Programa de Ingeniería de Sistemas

1 · HISTORIAS DE USUARIO

Cuatro historias de alto valor para el cliente

HU-01 · Autor de preguntas

Crea preguntas de selección múltiple con única respuesta.

- ✓ Al guardar valida la estructura y muestra todos los errores juntos.
- ✓ Queda en «Borrador» con código P-001, P-002…

HU-02 · Autor de preguntas

Envía sus borradores a «Pendiente de revisión».

- ✓ Solo el autor, y solo desde «Borrador».
- ✓ Cada estado se ve con su propio color.

HU-03 · Autor de preguntas

Lista sus preguntas con filtros y paginación.

- ✓ Filtra por estado, dificultad, tema, competencia y texto.
- ✓ Páginas de 5, 10 o 20 filas.

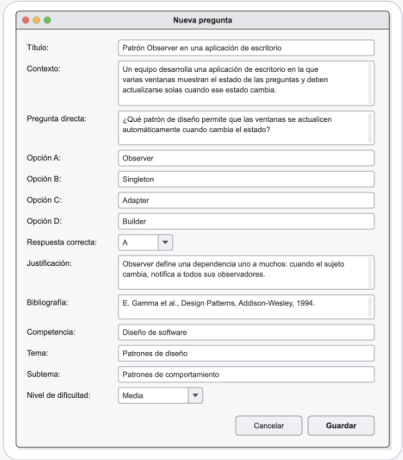
HU-04 · Administrador

Asigna uno o más revisores a las preguntas pendientes.

- ✓ No puede asignar al autor ni a usuarios inactivos.
- ✓ Cada revisor recibe un correo (o se simula en consola).

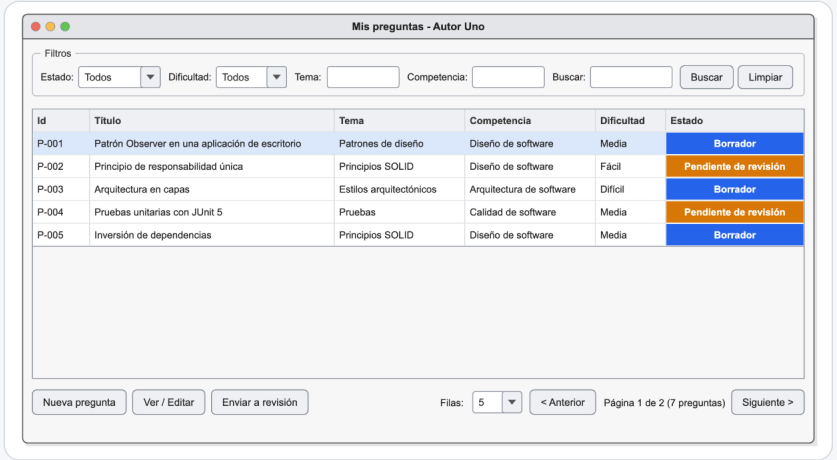
1 · PROTOTIPOS

Wireframes: formulario, listado y asignación



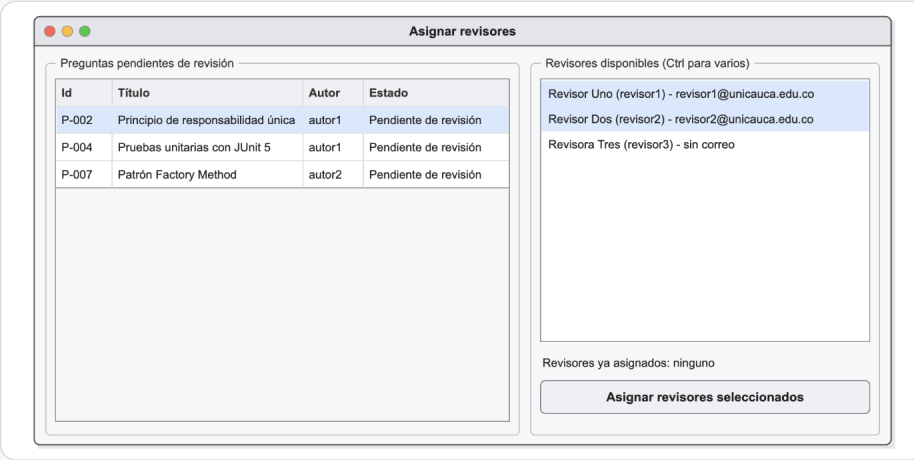
Formulario · HU-01

Respuesta y dificultad se escogen de una lista.



Mis preguntas · HU-02 y HU-03

Filtros arriba, estados con color y paginación abajo.



Asignar revisores · HU-04

Pendientes a la izquierda, revisores activos a la derecha.

1 · USABILIDAD

Test de usabilidad: pensar en voz alta

[N] personas

Cada persona hizo cinco tareas mientras decía en voz alta lo que pensaba. Nosotros solo observamos y anotamos.

1. Iniciar sesión como autor.
2. Crear una pregunta completa.
3. Buscar una pregunta con los filtros.
4. Enviar una pregunta a revisión.
5. Como administrador, asignar un revisor.

Lo que encontramos

- [Hallazgo 1]
- [Hallazgo 2]

Lo que cambiamos

- [Cambio 1]
- [Cambio 2]

2 · ATRIBUTOS DE CALIDAD

Modificabilidad: un canal de notificación nuevo

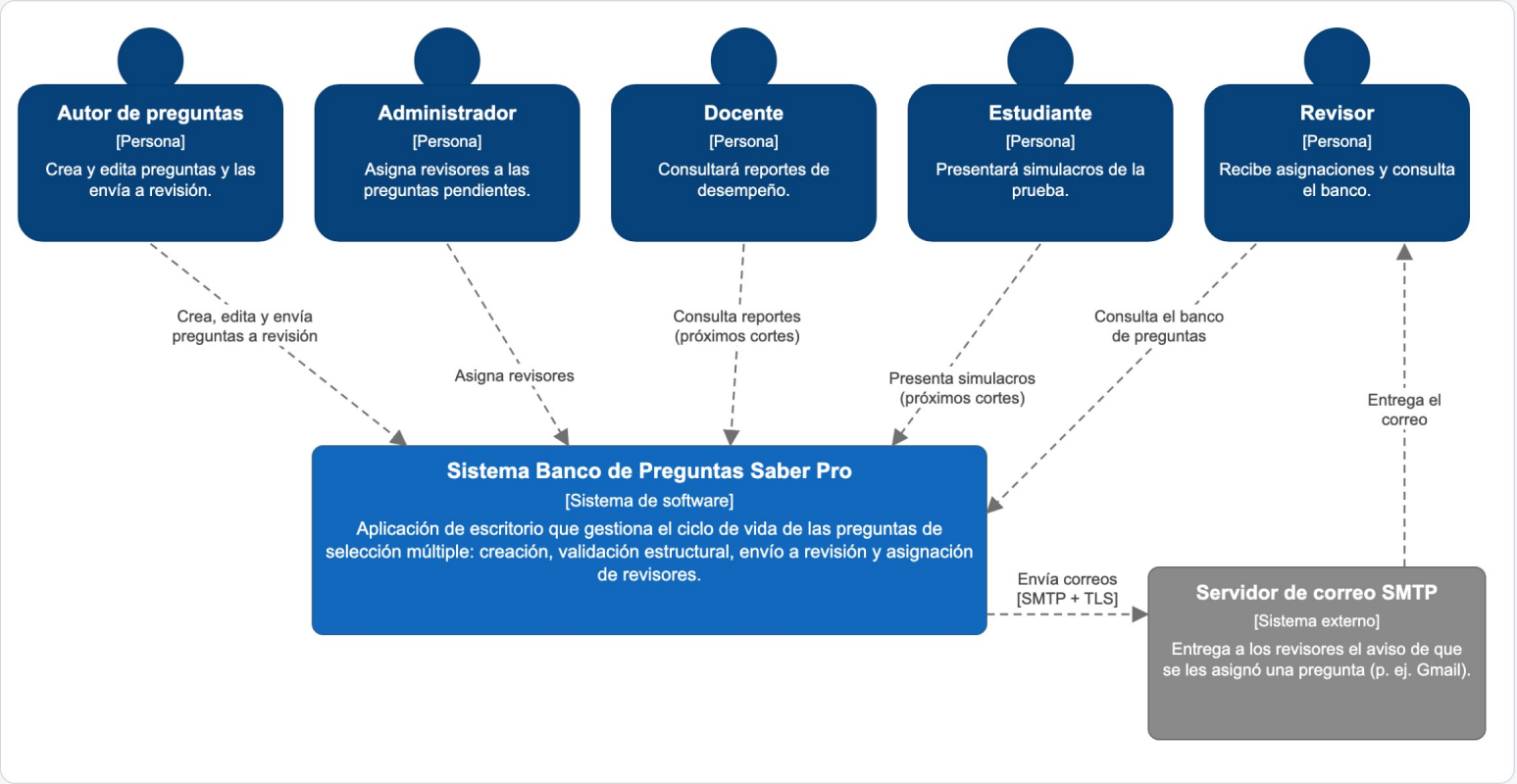
Estímulo	Piden avisar a los revisores por otro medio, por ejemplo WhatsApp o SMS.
Respuesta	Una clase nueva que implementa INotificador y un caso más en NotificadorFactory. ReviewService y las vistas no cambian.
Medida	2 clases tocadas (1 nueva + la fábrica) y 0 cambios en dominio, controladores y vistas.
Resultado esperado	El cambio queda aislado y las 110 pruebas siguen pasando.

```
// domain/INotificador.java
public interface INotificador {
  void notificarAsignacion(
    User revisor, Question pregunta);
}

// Ejemplo (aún no existe):
public class WhatsAppNotificador
    implements INotificador { ... }
```

¿Por qué importa? RNF-10 a RNF-14 piden modificabilidad, y en los cortes 2 y 3 este mismo código se refactoriza a microservicios y a arquitectura hexagonal.

Contexto: quién usa el sistema



5 roles

Autor, administrador y revisor ya tienen funciones.

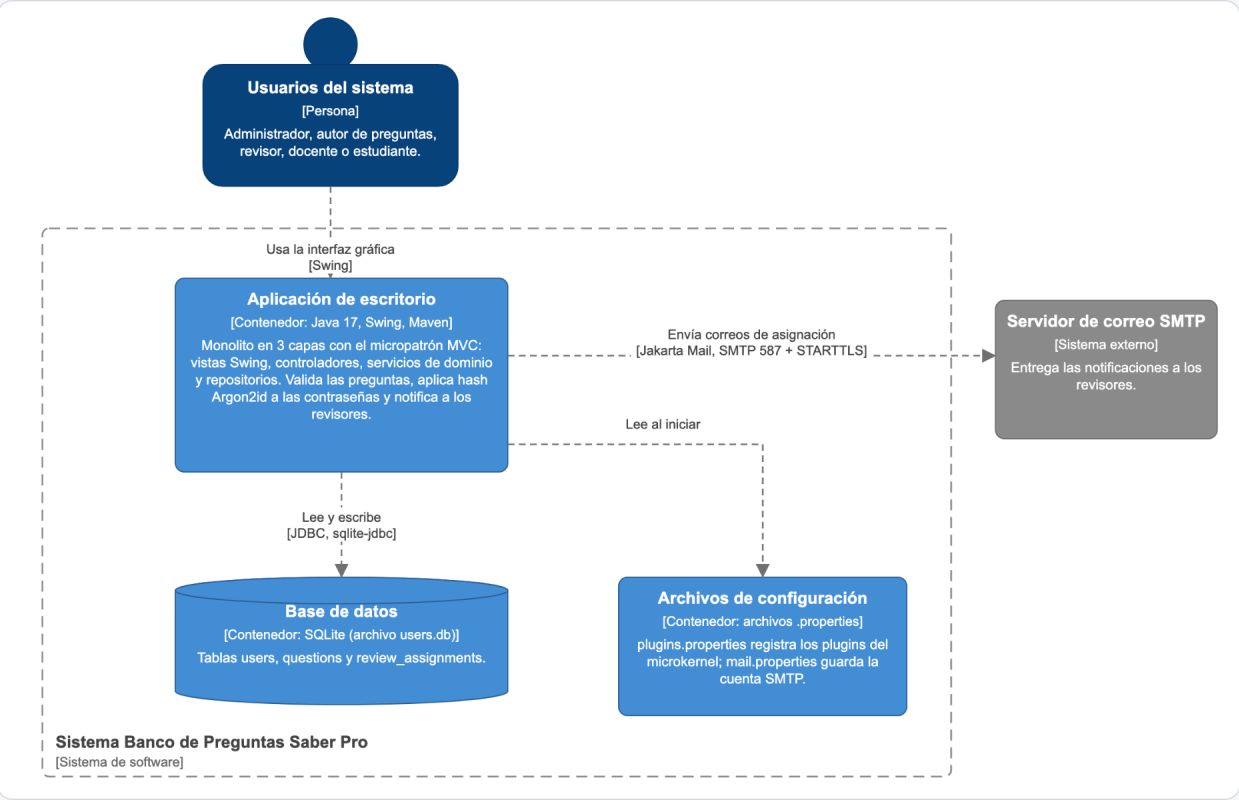
1 externo

Servidor SMTP para los correos.

Después

Docente y estudiante, en los cortes 2 y 3.

Contenedores: un monolito y su base de datos



Aplicación de escritorio
Java 17 y Swing, monolito en 3 capas.

SQLite · users.db
Tablas users, questions y review_assignments.

Archivos .properties
plugins.properties y mail.properties (este no se sube a Git).

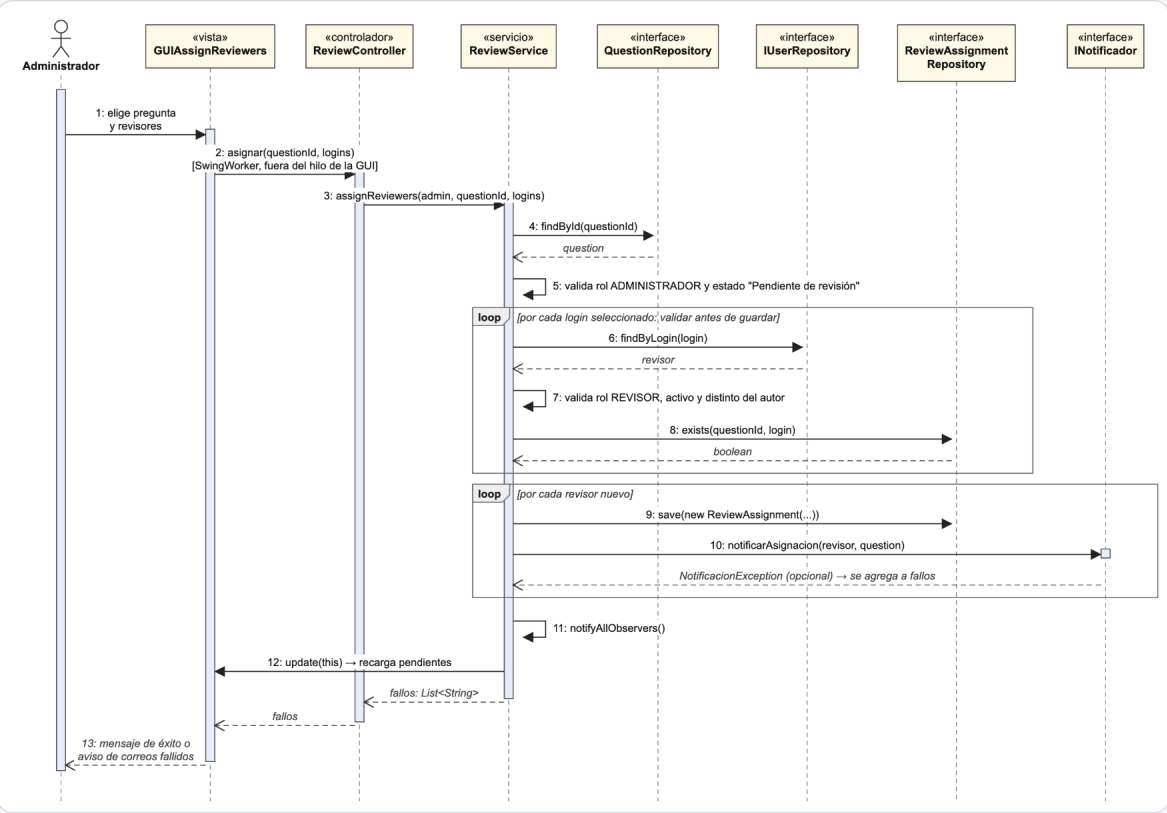
Servidor SMTP (externo)
Jakarta Mail con STARTTLS por el puerto 587.

Componentes: cada capa usa solo la de abajo

presentation Vista (Swing)	`LoginFrame` `DashboardFrame` `GUIQuestionForm` `GUIMyQuestions` `GUIAssignReviewers`
controller Controlador	`QuestionController` `ReviewController`
domain Modelo y reglas	`QuestionService` `ReviewService` `UserService` `QuestionValidator` `QuestionRepository` `ReviewAssignmentRepository` `INotificador`
access · infra Datos y correo	`SqliteQuestionRepository` `SqliteUserRepository` `SqliteReviewAssignmentRepository` `NotificadorFactory` `EmailNotificador` `ConsoleNotificador`

Borde punteado = interfaz del dominio. Las clases de access e infra las implementan, y AppContext las conecta.

Secuencia de la HU-04: asignar revisores



MVC

GUIAssignReviewers solo habla con ReviewController, y el controlador llama a ReviewService.

Dependencias hacia el dominio

ReviewService no conoce SQLite ni el correo: usa los repositorios e INotificador como interfaces.

Sin asignaciones a medias

Primero valida a todos los revisores. Si un correo falla, la asignación se conserva y se avisa.

Cada patrón resuelve un problema concreto

Repository

`domain/QuestionRepository`
`access/SqliteQuestionRepository`

El dominio no depende de SQLite: solo conoce la interfaz.

Observer

`infra/Subject · infra/Observer`

QuestionService y ReviewService avisan; GUIMyQuestions y GUIAssignReviewers se recargan solas.

Factory

`infra/NotificadorFactory.crear()`

Con mail.properties completo crea EmailNotificador; si no, ConsoleNotificador.

Microkernel + Pipes and Filters

`kernel/QuestionMicrokernel`
`plugins/multiplechoice + pipeline/`

Nuevos tipos de pregunta como plugins. Ojo: la HU-01 no usa este pipeline, la valida QuestionValidator.

Pruebas automáticas con JUnit 5

110

pruebas en 18 clases de prueba

```
$ mvn test
```

Se ejecutan en vivo: el resultado en verde queda en pantalla.

```
src/test/java/co/edu/unicauca/
  domain/      12 clases
  access/      4 clases
  controller/  1 clase
  pipeline/    1 clase
```

Qué cubren

- Entidades: Question, User y sus enumeraciones.
- Servicios: QuestionService, ReviewService y UserService.
- Validadores, filtros, paginación y repositorios SQLite.

Una regla del validador y una del servicio

QuestionValidatorTest

```
@Test
@DisplayName("Las opciones no pueden repetirse")
void opcionesRepetidas() {
  q.setDistractors(new QuestionDistractors(
      "Igual", "igual", "Tres", "Cuatro"));
  List<String> errores = validator.validate(q);
  assertEquals(1, errores.size());
  assertTrue(errores.get(0).contains("repetida"));
}
```

"Igual" e "igual" cuentan como repetidas: se compara sin mayúsculas.

QuestionServiceHistoriasTest

```
@Test
@DisplayName("HU2: el autor envía su borrador...")
void enviarARevision() {
  Question q = service.createQuestion(
      TestData.preguntaValida("autor1"));
  service.sendToReview(q.getId(), "autor1");
  assertEquals(Question.STATE_PENDIENTE_REVISION,
    service.getQuestionById(q.getId()).getEstado());
}
```

El servicio usa QuestionImplRepository, en memoria: la prueba no toca SQLite.

5 · REPOSITORIO Y TABLERO

Cómo trabajamos en equipo

GitHub

- ✓ Historial de commits de los tres integrantes.
- ✓ Ramas y pull requests.
- ✓ README con cómo ejecutar y los usuarios de prueba.

`[URL del repositorio]`

Tablero del Sprint 1

- ✓ Columnas: por hacer, en progreso y hecho.
- ✓ Una tarjeta por tarea de cada historia.
- ✓ Cada tarjeta con su responsable.

`[Enlace al tablero de Trello]`

Demo en vivo: cinco minutos, tres voces

Minuto	Quién	Qué se muestra
0:00	A	Login como autor, crear una pregunta, ver la validación fallar (campo vacío u opción repetida) y luego guardar.
1:15	A	Listado: estados con colores, paginación y filtros. Enviar la pregunta a «Pendiente de revisión».
2:15	B	Login como administrador, asignar revisores y mostrar la notificación (correo o consola).
3:15	C	Código que evidencia la arquitectura: vista → controller → servicio, interfaz del repositorio, AppContext, Factory y Observer.
4:45	Todos	Cierre en 15 segundos.

La vista llama al controller y este al servicio

presentation/GUIQuestionForm

```
private void guardar() {
  Question q = leerFormulario();
  if (existing == null) {
    Question creada = controller.crear(q);
    ...
```

controller/QuestionController

```
public Question crear(Question question) {
  question.setAutor(author.getLogin());
  return service.createQuestion(question);
}
```

domain/QuestionService

```
public Question createQuestion(Question question) {
  ...
  validator.validateOrThrow(question);
  question.setId(nextId());
  question.setEstado(Question.STATE_BORRADOR);
  if (!repository.save(question)) {
    throw new IllegalStateException(...);
  }
  notifyAllObservers();
  return question;
}
```

La vista no valida ni guarda; el servicio no sabe que existe Swing.

El servicio depende de una interfaz

```java
// domain/QuestionRepository.java
public interface QuestionRepository {
  List<Question> findAll();
  Question findById(String id);
  boolean save(Question question);
  boolean updateState(String id, String newState);
  ...
}

// domain/QuestionService.java
private final QuestionRepository repository;
public QuestionService(
    QuestionRepository repository) {
  ...
  this.repository = repository;
}
```

```java
// AppContext.java: raíz de composición
SqliteDatabase db = new SqliteDatabase();
QuestionRepository questionRepository =
    new SqliteQuestionRepository(db);
INotificador notificador =
    NotificadorFactory.crear();
this.questionService =
    new QuestionService(questionRepository);
this.reviewService = new ReviewService(
    questionRepository, assignmentRepository,
    userRepository, notificador);
```

Solo AppContext conoce SQLite y el correo. Para cambiar de base de datos se cambian estas líneas, no los servicios.

NotificadorFactory y Subject / Observer

infra/NotificadorFactory

```
public static INotificador crear(Path configFile) {
  if (!Files.exists(configFile)) {
    return new ConsoleNotificador();
  }
  // ... lee mail.properties en props
  if (isBlank(props.getProperty("mail.username"))
    || isBlank(props.getProperty("mail.password"))
    || isBlank(props.getProperty("mail.from"))) {
    return new ConsoleNotificador();
  }
  return new EmailNotificador(props);
}
```

infra/Subject · presentation/GUIMyQuestions

```
// Subject: lo extienden los servicios
public void notifyAllObservers() {
  for (Observer observer : observers) {
    observer.update(this);
  }
}

// GUIMyQuestions implementa Observer
@Override
public void update(Object obj) {
  cargar();
}
```

El resto del sistema solo conoce INotificador, y los servicios solo conocen la interfaz Observer, nunca las ventanas.

CIERRE

En resumen

4 historias

Crear, enviar a revisión, listar y asignar revisores, funcionando.

3 capas + MVC

El dominio no depende de SQLite ni del correo.

Patrones con propósito

Repository, Observer, Factory, Microkernel y Pipes and Filters.

110 pruebas

Sobre las entidades y los servicios del dominio.

Lo que sigue: microservicios y eventos en el corte 2; arquitectura hexagonal en el corte 3. ¡Gracias!